Implementation and stress report · Sofuu 0.3.0-beta

Engineering report · 2026-10-29

Universal Plugin & Extensibility System

Sofuu 0.3.0-beta · implementation and stress report · 29 October 2026

1. Executive summary

PLUGIN_PLAN.md described five plugin categories and four delivery phases. All of it is implemented, wired into the binary and verified:

Plan phase	Delivered	Evidence
1. Engine hardening, modularise `plugins.js`, TypeScript support	`plugins.js` rebuilt as eleven modules, 3,230 lines (was 1,294); scanner-based TS stripper	14/14 stripper fuzz cases pass; 4/4 cookbook `.ts` plugins validate
2. Native Ollama + llama.cpp adapters, `sofuu --provider`	new `providers.js` driver (1,099 lines) with 5 adapters; `Provider::LlamaCpp` in Rust; `-p ollama|llamacpp`	31/31 adapter harness checks; 6/6 against the real engine
3. `sofuu.d.ts` SDK typings	1,196 lines, 99 declarations, zero bare `any`	`tsc --strict`: 0 errors; runtime/typings parity gate 5/5
4. Cookbook plugins	four runnable plugins in `plugins/`	each loads, registers and runs through the real CLI

Stress result: 25 of 25 adversarial cases pass (all five categories, full run, 6.7 s). Six real defects were found during the build-out and fixed — they are written up in §5 because they are the most useful part of this report: a policy denial used to *trip* the circuit breaker (hiding `E_DENIED` behind `E_CIRCUIT_OPEN`), the TypeScript stripper desynchronised on regex literals containing quotes and separately swallowed function bodies, a plugin that overwrote a built-in looked like it had registered nothing, a stream-only provider was rejected, and `plugin run` printed “ok” instead of the payload.

A clause-by-clause audit against the shipped binary is in **§12**: every requirement of PLUGIN_PLAN.md is verified except three, which are named there (pluggable-KV *replacement*, incremental streaming from a JS adapter, and OpenAPI → micro-Zod which emits JSON Schema through the same validator). The audit is what the plan’s own examples are judged by — including `z.enum([...]).default('celsius')`, the JSON-RPC `-32602` error data, `useOpenAPI` taking a URL **or** a path, and `sofuu --provider ollama --model llama3.2` without a subcommand.

The binary is **5,688,848 bytes** against a hard cap of **5,767,168** — 78,320 bytes (1.4 %) of headroom. Growth for the whole feature is +33,040 bytes, almost all of it the two new bytecode snapshots.

2. Scope delivered

2.1 The five categories

#	Category	Factory	What it registers	Shield coverage
1	Action	`defineTool`	LLM-callable tool, micro-Zod or JSON Schema in/out	S1–S5
2	Inference	`defineProvider`	model backend adapter	S1, S3, S4, S5
3	Persistence	`defineStorage`, `defineVectorStore`	KV store, vector index, dual-write mirror	S1, S5
4	Lifecycle	`defineService`	`onStart`, `onTurnStart`, `onToolCall`, `onToolResult`, `onTurnEnd`, `onStop`	isolated, time-boxed
5	Description	`useOpenAPI`, `ingestMcp`	REST operations / MCP tools as 5-shield tools	S1–S5

2.2 The runtime surface

`sofuu.plugins` exposes 36 functions and 9 objects. The public contract:

```
defineTool  defineProvider  defineStorage  defineVectorStore  defineService
useOpenAPI  ingestMcp      discover       discoverAll        loadManifest
list  invoke  registerProvider  getProvider  resolveProvider  listProviders
completeVia  streamVia     attachVectorStore  attachStorage  mirrorStats
flushMirror  emitTurnStart  emitTurnEnd  emitToolCall  emitToolResult
startServices  stopServices  capabilities  resetCircuit  circuitState
cliList  cliValidate  cliRun  cliTest  cliInit
```

Eleven internal modules — schema, security, breaker, watchdog, registry, provider, store, service, openapi, discovery, cli — with every 0.2.0 public name preserved.

2.3 The CLI contract from plan §3

Command	Plan requirement	Delivered
`plugin list`	table with name, category, tool/provider count, status	`NAME · CATEGORY · COUNT · CIRCUIT · TIMEOUT · DESCRIPTION`, where COUNT is how many registrations came from that plugin file (3 for a file exporting three tools, 1 for a hand-written plugin), and CIRCUIT is the status
`plugin init <name> --category=`	typed starter in `./plugins/<name>.ts` with a Zod schema	six templates — tool, provider, store, vectorstore, service, openapi — each a loadable `.ts` file that validates
`plugin test <tool> '<json>'`	isolated execution, latency and returned JSON	resets the target's breaker first (so an earlier failure cannot mask a fresh test), prints `ok/true|false, code, latency`, then the **payload** — a tool returning `{ok:true,...}` shows its own fields, not the word "ok"
`plugin validate <file.ts>`	syntax, exports, missing schemas, unhandled rejections	parses, strips, evaluates, lists what registered, and **warns when a tool declares no schema** (inert Shield 1); a broken file is reported per-file without aborting discovery

3. Architecture

3.1 Files

File	Lines	Change	Role
`src/js/plugins.js`	3295	rewritten (+3,348 / −731 with the rest)	the engine: five categories, five shields, discovery, CLI helpers, TS stripper, YAML parser
`src/js/providers.js`	1,099	new	ollama, llamacpp, openai, lmstudio, vllm adapters
`sofuu.d.ts`	1,196	new	SDK typings, shipped to npm
`plugins/*.ts`	1,224	new	cookbook: `fetch_web`, `sqlite_query`, `ollama_provider`, `token_logger`
`docs/PLUGINS.md`	267	new	user guide
`scripts/stress/plugin_stress.js`	560	new	25-case adversarial gate
`scripts/stress/plugin_types_check.js`	147	new	runtime/typings parity + cookbook strip gate
`crates/.../rt/ai.rs`	+60	edit	`Provider::LlamaCpp`, keyless path, `/embeddings` URL
`crates/.../rlm/js_api.rs`	+11	edit	evaluate the providers driver
`crates/.../shipped.rs`	+12	edit	embed `providers.bc.gz`
`crates/.../main.rs`	50	edit	`plugin test`, `init --category` (6), `-p ollama|llamacpp`
`crates/.../chat.rs`	small	edit	seven valid providers, 32k ctx for local engines

Everything else — `agent.js` (+146), `chat.js`, `README.md`, `CHANGELOG.md`, `Makefile` (`plugin-check`, `plugin-stress`), `npm/package.json` (`types`) — is described where it belongs below. Total: **18 files changed, +3,063 / −731**.

3.2 Dependencies

The engine adds **no dependency**: micro-Zod, the YAML 1.2 subset parser (anchors excluded, flow collections included) and the TypeScript stripper are all in-file, because the binary cap does not allow a parser library and the governance rules forbid new vendored code.

4. The five shields

#	Shield	Code	Implementation notes
1	Schema	`E_SCHEMA`	Validated before `run()` and, when declared, after it. A malformed *schema* is rejected at define time instead of degrading to “accept anything”. Soft failures (`{ok:false}`) bypass output validation.
2	Policy & network	`E_DENIED`	`permissions.net` allowlist, private-IP/SSRF guard (decimal and hex-encoded loopback included), `file://`, `data:`, `ftp://`, credential-bearing URLs, jailed `ctx.fs`.
3	Circuit breaker	`E_CIRCUIT_OPEN`	CLOSED/OPEN/HALF_OPEN, 5 failures / 30 s default, single-trial admission under a 10-way concurrent burst.
4	Watchdog	`E_TIMEOUT`	Per-call deadline; abort signal delivered through `ctx.signal`. 100/100 hung calls time out at ~62 ms on a 60 ms budget.
5	Panic guard	`E_TOOL`	Sync throw, rejected promise, `throw null`, circular return, BigInt return — all normalised. The host never crashes.

Two codes join the taxonomy: `E_CAPABILITY` (a plugin asks for something this build does not have) and `E_BACKPRESSURE` (a buffered adapter hit its 8 MB cap).

5. Defects found and fixed during the build

These are the parts worth reading, because they were found by adversarial testing rather than by review.

5.1 A policy denial tripped the circuit breaker

`defineTool` counted every failure — including `E_DENIED` — toward the breaker. The stress case fired eleven hostile targets at one tool; the first five returned `E_DENIED` (correct), the breaker opened, and the remaining six returned `E_CIRCUIT_OPEN`. The security control still held, but the *reason* was replaced by a symptom, which is exactly what makes an incident report useless.

Fix: `E_DENIED` no longer records a failure. The case now reports **16/16 denied**.

5.2 The TypeScript stripper desynchronised on regex literals

`s.replace(/<a[^>]*href="([^"]*)"[^>]*>/gi, function (all: string, ...) {})` broke the scanner: it saw the quote inside the regular expression, entered “string mode”, and skipped the parameter list — leaving the annotations in place and producing invalid JavaScript.

Fix: a regex-literal skipper that engages only where a `/` cannot be division (previous significant character is an operator or an opening bracket), plus a `prevSig` tracker in the annotation pass. This is the same class of bug that made the previous stripper eat `{ units: defaultUnits }`; both are now covered by fuzz cases.

5.3 A type atom swallowed the function body

`function parseColumns(raw: string): string[] {` — `tsTypeEnd` matched `string[]`, then treated the `{` of the **body** as an object type literal, consumed the whole body, failed to find a terminator, and returned “not a type”. Every top-level function in a plugin therefore kept its annotations.

Fix: a `{` after a matched type atom terminates the type; only a leading `{` can be an object type.

5.4 A plugin that overwrites a built-in looked like “registered nothing”

Discovery diffed registry *names*. The cookbook `ollama_provider.ts` registers a provider named `ollama`, which the shipped adapter had already registered — so the key existed, the diff was empty, and the file was reported broken.

Fix: snapshots compare object identity, and a factory’s returned spec counts as a registration. `plugin validate` now reports `provider:ollama`.

5.5 `plugin run` threw the payload away

A tool that returns `{ok:true, ...}` *is* the envelope (the engine passes it through untouched), so `res.data` was undefined and the CLI printed the envelope’s `text` field — the word “ok”. `sofuu plugin test hello_manifest` answered “ok” while the tool had returned `{from:"manifest"}`. `cliRun` now extracts the payload from the envelope itself.

5.6 A stream-only provider was rejected

`assertProviderSpec` demanded `complete()`. A provider that only implements `stream()` is legal by the plan’s contract, and the stress case caught the contradiction. Both entry points are now accepted.

6. Provider adapters and the Rust side

Adapter	Wire	Key	Notes
`ollama`	`POST /api/chat`, NDJSON	none	`options.temperature`, `num_predict`, `format`, tool calls, `prompt_eval_count`/`eval_count` usage
`llamacpp`	`{base}/v1/chat/completions`, SSE	none	GBNF `grammar`, `response_format`
`openai`	`{base}/v1/chat/completions`, SSE	required	Bearer; missing key is `E_DENIED` *before* a request is made
`lmstudio`	OpenAI wire	none	`127.0.0.1:1234`
`vllm`	OpenAI wire	none	`127.0.0.1:8000`

One wire parser serves the four OpenAI-shaped adapters. `usage` is normalised to `{promptTokens, completionTokens, totalTokens, raw}` for every wire shape — snake_case, camelCase and Ollama counters — because the previous adapter returned the raw wire keys and the engine's contract check caught it.

Rust gains `Provider::LlamaCpp` with three deliberate properties: it is **keyless** (without the explicit arm it would have inherited `OPENAI_API_KEY` and sent a cloud key to `127.0.0.1:8080`), the `.gguf` in-process heuristic is gated off for it (so an explicit llama.cpp call is never hijacked into local GGUF inference), and embeddings resolve to `/v1/embeddings`.

7. TypeScript without a compiler

A `.ts` plugin is stripped textually and evaluated; the runtime stays free of a compiler and the binary stays small. The stripper is a scanner, not a regex pile: it handles imports/exports, `interface`/`namespace`/`module` blocks (brace-matched), type aliases, `enum` → frozen object, `as`/`satisfies`, generics on functions and arrows, parameter/return annotations decided by the enclosing construct, optional parameters, parameter properties and class field annotations.

Not supported	Why
`new Promise<never>(...)`	Parses as comparisons once stripped (documented; use a typed variable)
Decorators	No transform
`abstract`, `implements`, overload signatures	Not removed
Single-line class bodies with annotated fields	Member rules are line-oriented

The design rule is asymmetry of failure: when a construct cannot be classified it is **left alone**, so a mistake fails loudly at parse time instead of silently deleting a value. Editor support is real `tsc` — `sofuu.d.ts` ships with the package.

8. Verification

8.1 Adversarial stress — 25/25

```
=== Cat 1: defineTool / 5 shields ===
  PASS S1.1 schema rejects malformed args under concurrency  [bad=1000 good=1000]  89ms
  PASS S1.2 output schema is enforced  [E_SCHEMA]
  PASS S1.3 a garbage schema is rejected at define time  [E_SCHEMA]
  PASS S2.1 SSRF matrix: every hostile target is denied  [16/16 denied]
  PASS S2.2 an explicit net allowlist admits the loopback server
  PASS S3.1 circuit trips after threshold and fails fast  [opened after 5 calls]
  PASS S3.2 HALF_OPEN admits exactly one trial under a concurrent burst  [trials=1 fastfail=9]
  PASS S4.1 watchdog fires near the deadline (100 runs)  [timeouts=100 avg=62ms]
  PASS S5.1 panic guard survives every JS failure shape
  PASS S6.1 400-way concurrent mixed load keeps envelopes intact  [n=400]
=== Cat 2: defineProvider / completeVia / streamVia ===
  PASS P1.1 a provider completes and reports usage
  PASS P1.2 a throwing provider is normalised to E_TOOL
  PASS P1.3 provider streaming replays deltas in order  [deltas=3]
  PASS P1.4 150 concurrent provider calls stay isolated  [n=150]
  PASS P1.5 provider registry is introspectable  [providers=8]
=== Cat 3: defineVectorStore / dual-write mirror ===
  PASS M1.1 local write is synchronous, remote is write-behind  [local=2001 written=2000 failed=1]
=== Cat 4: defineService hooks ===
  PASS L1.1 onTurnStart rewrites the prompt
  PASS L1.2 onToolCall blocks a dangerous call and mutates args
  PASS L1.3 a throwing service cannot break the others
  PASS L1.4 500 hook dispatches stay under budget  [0.005ms/call]
=== Cat 5: useOpenAPI / MCP ===
  PASS O1.1 YAML spec with refs generates callable tools  [tools=2]
  PASS O1.2 a malformed spec is refused with E_SCHEMA
  PASS O1.3 MCP ingestion degrades cleanly with no server  [E_CAPABILITY]
=== Phase 1: stripTs fuzz ===
  PASS T1.1 every TS sample strips to valid JS or throws typed  [parsed=14]
  PASS T1.2 plain JS passes through unchanged

cases=25 PASS=25 FAIL=0 SKIP=0 wall=6706ms
```

8.2 Engine and typings parity — 5/5

```
PASS runtime surface is complete  [36 functions + 9 objects]
PASS sofuu.d.ts declares the documented surface  [22 declarations]
PASS sofuu.d.ts is structurally sound  [125 blocks, 99 exports]
PASS every runtime function is named in the typings  [36 checked]
PASS cookbook plugins strip to parseable JavaScript  [4 plugins]
```

8.3 CLI, end to end

```
$ sofuu plugin validate plugins/fetch_web.ts
Plugin plugins/fetch_web.ts is valid (tool:fetch_web)
$ sofuu plugin validate plugins/ollama_provider.ts
Plugin plugins/ollama_provider.ts is valid (provider:ollama)
$ sofuu plugin validate plugins/sqlite_query.ts
Plugin plugins/sqlite_query.ts is valid (tool:sqlite_query)
$ sofuu plugin validate plugins/token_logger.ts
Plugin plugins/token_logger.ts is valid (service:token_logger)

$ sofuu plugin test sqlite_query '{"sql":"SELECT count(*) FROM notes"}'
ok=true code=OK latency=118ms
{ "engine": "qtsq-structured-store", ... }

$ sofuu plugin test sqlite_query 'not json'
Error: invalid JSON arguments: unexpected token: 'not'

$ sofuu plugin run nope '{}'
ok=false code=E_CAPABILITY latency=62ms
Tool Error [E_CAPABILITY]: Plugin tool "nope" not found
```

`sofuu plugin list` reports 8 registrations — 2 cookbook tools, 1 service and 5 provider adapters — with per-plugin circuit state and a COUNT column:

```
NAME                CATEGORY    COUNT  CIRCUIT     TIMEOUT   DESCRIPTION
fetch_web           tool        1      CLOSED      5000ms    Fetch an http(s) URL and return clean
text
sqlite_query        tool        1      CLOSED      10000ms   Read-only SELECT against the QTSQ
structured store
ollama              provider    1      CLOSED      120000ms  Local Ollama bridge (/api/chat) ...
...
token_logger        service     1      CLOSED      30000ms   Appends one JSONL token line per turn

$ sofuu plugin validate plugins/multi.ts        # a file exporting three tools
  warning: no_schema_tool: no schema declared: arguments are not validated
Plugin plugins/multi.ts is valid (tool:multi_one, tool:multi_two, tool:no_schema_tool)
```

Discovery tags every registration with the file that produced it, so COUNT is real: a file exporting three tools shows `3`, an OpenAPI namespace shows the whole family, and an overwritten built-in keeps its origin.

8.4 Regressions

Suite	Before	After
Contract conformance	51 PASS · 0 FAIL · 1 BLOCKED	**51 PASS · 0 FAIL · 1 BLOCKED** (identical; T4.2 still needs macOS+Android runners)
JS test suite	57 passed · 3 failed · 1 skipped (61)	**57 passed · 3 failed · 1 skipped (61)** (identical; same three files)

Both suites are unchanged by this work. The three JS failures (`priority1_test.js`, `abi_script_test.js`, `brain_auth_e2e.sh`) are pre-existing on this machine — the first two are ABI-extension acknowledgement gates, the third needs a brain auth endpoint — and none of them touch the plugin driver. Contract T4.2 remains blocked because it needs macOS **and** Android runners in the same job.

8.5 Binary budget

	Bytes
Before	5,655,808
After	5,688,848
Delta	+33,040
Cap (`SIZE_LIMIT`)	5,767,168
Headroom	**78,320 (1.4 %)**

Bytecode snapshots dominate the delta: `plugins.bc.gz` 15,734 → 37,045, `providers.bc.gz` +10,407, `agent.bc.gz` +1,398.

9. Limits, stated plainly

- Plugin code runs **in-process**. The shields contain faults, not malice; a plugin that allocates unbounded memory can still hurt the process. Third-party plugins are third-party code.
- **JS provider streaming is buffered.** `streamVia` reads the whole body, then replays deltas in order and emits one `done`. The in-process `fetch` exposes no `ReadableStream`, so incremental delivery stays on the Rust native path (`sofuu chat -p ollama` streams token by token). Adapters advertise `streaming: 'buffered'`; a body over 8 MB fails with `E_BACKPRESSURE`.
- The TypeScript stripper is not `tsc` (§7). Unsupported constructs fail at parse time by design.
- `useOpenAPI` implements no authentication flow — pass headers, keep secrets in the environment. It never fetches a remote document; you supply the spec.
- MCP ingestion wraps tools with the same shields, but the MCP transport itself is the host's; without one it degrades to `E_CAPABILITY`.
- `detectProfile` still labels any `127.0.0.1` endpoint as `local`, so a llama-server on :8080 displays as local. Cosmetic, out of scope, noted.
- No SQLite exists in this runtime: the cookbook's `sqlite_query` exposes the native QTSQ structured store through a documented read-only SELECT subset, and says so in its header.

10. Reproducing

```
bash scripts/gen-driver-bc.sh   # src/js/*.js -> *.bc + *.bc.gz (needs ./sofuu)
make -j8                        # embeds the snapshots
make bc-check                   # snapshots are current
make size-check                 # binary within the 5,767,168-byte cap
make plugin-check               # typings/runtime parity + cookbook strip
make plugin-stress              # 25 adversarial cases
./sofuu plugin list             # 8 registrations, circuits CLOSED
```

12. Plan conformance audit

Every clause of PLUGIN_PLAN.md, checked against the shipped binary rather than against the source. "Verified" means a command in this session produced the evidence quoted.

12.1 Category 1 — `defineTool`

Plan clause	Status	Evidence
The §2 example compiles and runs as written (incl. `z.enum([...]).default('celsius')`, `describe`, `timeoutMs`, `requiresApproval`)	**Verified**	probe 1: `units=celsius` when omitted, `fahrenheit` when passed
Shield 1 rejects malformed arguments "with typed JSON-RPC `-32602` error data"	**Verified**	probe 2: `E_SCHEMA, jsonrpc:-32602, data:{tool,code,path:"city"}`
Shield 2 policy/SSRF (`E_DENIED`)	**Verified**	stress S2.1: 16/16 hostile targets denied, loopback allowed only by `permissions.net`
Shield 3 trips after 5 consecutive failures (`E_CIRCUIT_OPEN`)	**Verified**	stress S3.1 (`opened after 5 calls`), S3.2 (one trial under a 10-way burst)
Shield 4 hard deadline + `AbortSignal` (`E_TIMEOUT`)	**Verified**	stress S4.1: 100/100 timeouts at 62 ms
Shield 5 panic guard (`E_TOOL`)	**Verified**	stress S5.1: sync throw, rejected promise, `throw null`, circular, BigInt

12.2 Category 2 — `defineProvider`

Plan clause	Status	Evidence
The §2 example shape: destructured `{ messages, model, stream, tools }` and `this.baseUrl`	**Verified**	probe: `base=http://127.0.0.1:9 model=llama3.2 msgs=1` (`this` is the spec)
Ollama adapter, `/api/chat`, tool calling	**Verified**	adapters harness 31/31; stress P1.1–P1.5
Ollama **streaming**	**Partial by platform**	JS adapters replay a buffered body (`streaming:'buffered'`, 8 MB cap → `E_BACKPRESSURE`); incremental streaming exists only on the native Rust path (`sofuu chat -p ollama`). The in-process `fetch` exposes no `ReadableStream`.
llama.cpp adapter at `localhost:8080/v1/chat/completions` with grammar constraints	**Verified**	adapter harness: SSE, `grammar`, `response_format`; Rust `Provider::LlamaCpp` keyless
OpenAI / vLLM / LM Studio / cloud adapter	**Verified**	one shared OpenAI-wire parser, `lmstudio` (:1234) and `vllm` (:8000) aliases
`sofuu --provider ollama --model llama3.2`	**Verified**	bare CLI now enters chat (`ollama/llama3.2` in the session header); `-h`/`--version` still handled

12.3 Category 3 — storage and vector stores

Plan clause	Status	Evidence
`defineVectorStore({ init, upsert({id,vector,text,metadata}), query({vector,limit,filter}) })`	**Verified**	cookbook + stress M1.1; `P.STORES` registry, mirror queue
Dual-write / mirror mode (fast local write, async remote sync)	**Verified**	stress M1.1: `local=2001 written=2000 failed=1 localMs=7` — the remote failure is counted, never blocks the local write
Pluggable KV: *replace* the file-based store with Redis/SQLite	**Partial**	`defineStorage` + `attachStorage` implement the adapter **and mirror** path; the runtime's own QTSQ KV is not replaceable (no primary-read routing). Mirroring is done; replacement is not.

12.4 Category 4 — services

Plan clause	Status	Evidence
`onStart` / `onTurnStart` / `onToolCall` / `onTurnEnd` / `onStop`	**Verified**	stress L1.1–L1.4; agent seam wired in `agent.js`
`onTurnStart` may **mutate** `turn.prompt` in place (plan's PII-redaction style)	**Verified**	probe: an in-place rewrite is visible to the caller
`onToolCall` throws → the tool call is blocked	**Verified**	stress L1.2 (`E_DENIED` travels back as a tool result)
A throwing service cannot break the loop	**Verified**	stress L1.3
`onTurnEnd({ response, usage })` for token accounting	**Verified**	cookbook `token_logger.ts` writes JSONL per turn

12.5 Category 5 — protocol bridging

Plan clause	Status	Evidence
`useOpenAPI(specUrl, options)` — “pass a URL or local path”	**Verified**	probe 3: local path → `tools=1`, live call returned `{"greeting":"hello path"}`; probe 4: `http://127.0.0.1:13399/openapi.json` → live call returned `{"greeting":"hello url"}`
Parses `paths` for GET/POST/PUT/DELETE, parameters and request bodies	**Verified**	stress O1.1 (YAML + `$ref` → 2 callable tools); `filter` option honoured (probe 6)
“Extracts parameters into **micro-Zod** schemas”	**Done differently**	generated tools carry **JSON Schema**, which `defineTool` validates with the same Shield 1 code path. Equivalent enforcement, different representation; documented in §9.
OpenAPI 3.0/3.1 **and** Swagger 2.0	**Verified**	version detection + `host/basePath` synthesis + body-parameter mapping
MCP ingestion over **stdio**	**Verified**	stress O1.3 degrades to `E_CAPABILITY` with no server; wrapper used by the CLI
MCP ingestion over **SSE/HTTP**	**Wired, not live-tested**	`ingestMcp(server, { url })` passes `url`+`transport` to the host client, whose HTTP/SSE transport already exists (`tools.js`, `accept: text/event-stream`). A dead endpoint returns `E_CAPABILITY` (probe 7). No live SSE MCP server was run in this session.

12.6 §3 DX and §4 phases

Plan clause	Status	Evidence
`plugin list` with name, category, count, status	**Verified**	COUNT column (3 for a file exporting three tools), CIRCUIT column
`plugin init <name> --category=<kind>` scaffolds typed `.ts` with a Zod schema	**Verified**	six templates; each validates
`plugin test` isolated, latency + returned JSON	**Verified**	breaker reset before the call; payload printed, not the envelope
`plugin validate` flags bad exports and **missing schemas**	**Verified**	`warning: no_schema_tool: no schema declared`
Phase 1: modularise, TS stripping, typed errors	**Verified**	11 modules, 14/14 stripper fuzz, typed taxonomy
Phase 2: adapters + CLI flag	**Verified**	§12.2
Phase 3: `sofuu.d.ts` with the six exported factories	**Verified**	`tsc --strict` 0 errors; 99 declarations; parity gate 5/5
Phase 4: four cookbook plugins	**Verified**	`plugins/` — all four validate and load

12.7 What is not fully implemented

Three items, stated plainly:

1. **Pluggable KV replacement** (Cat 3) — mirroring works, primary replacement does not. This needs the runtime's storage reads to route through the plugin, which is a QTSQ-internal change rather than a plugin-system one.
2. **Incremental streaming from a JS provider adapter** (Cat 2) — buffered replay only; the native path streams. Platform limitation of the embedded `fetch`, not an omission.
3. **OpenAPI → micro-Zod** — JSON Schema is emitted instead (same validator).

Everything else in the plan is implemented and verified above.

13. What to read next

- Guide: `docs/PLUGINS.md`
- Runnable examples: `plugins/README.md`
- Typings: `sofuu.d.ts`
- Gates: `scripts/stress/plugin_stress.js`, `scripts/stress/plugin_types_check.js`
- Ship notes: `CHANGELOG.md` (Universal Plugin & Extensibility System)